



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №5
по курсу «Языки и методы программирования»
«Монады в языке Java»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Приобретение навыков использования монад `Optional` и `Stream` в программах на языке Java.

2 Задание

Во время выполнения лабораторной работы требуется разработать на языке Java один из классов, перечисленных в таблице, которая приведена ниже. В каждом классе нужно реализовать по крайней мере два метода: первый метод должен возвращать `Stream`, а второй – `Optional`. Операции, выполняемые каждым методом, указаны в вариантах задания. В методе `main` вспомогательного класса `Test` нужно продемонстрировать работоспособность разработанного класса, осуществив группировку содержимого потока, возвращаемого первым методом, с помощью группирующего коллектора. В исходном коде (включая класс `Test`) запрещено использовать циклы и рекурсию. Варианты 21, 23

3 Результаты

Исходный код программы представлен в 1, 2, 3, 4, 5, 6, 7, 8, 9.

Листинг 1: Файл

```
1 package st;
2
3 class Product {
4     int cost, count;
5     String name;
6
7     Product(String name, int count, int cost) {
8         this.name = name;
9         this.count = count;
10        this.cost = cost;
11    }
12 }
```

Листинг 2: Файл

```
1 package st;
2
```

```

3 import java.util.*;
4 import java.util.stream.Stream;
5
6 class ProductTable {
7     HashMap<String, Product> Table;
8     int total;
9
10    ProductTable() {
11        Table = new HashMap<>();
12        total = 0;
13    }
14
15    void add(Product p) {
16        Table.put(p.name, p);
17        total += p.count;
18    }
19
20    void add(String name, int cost, int count) {
21        Table.put(name, new Product(name, count, cost));
22        total += count;
23    }
24
25    public Stream<String> nameStream(int v) {
26        return Table.values().stream()
27            .filter(p -> p.count * p.cost > v)
28            .map(p -> p.name);
29    }
30
31    public Optional<Product> getProduct() {
32        return Table.values().stream()
33            .filter(p -> p.count * 2 > total)
34            .findFirst();
35    }
36 }

```

Листинг 3: Файл

```

1 package st;
2
3 import java.util.*;
4 import java.util.stream.Collectors;
5
6 public class Test {
7     public static void main(String[] args) {
8         ProductTable t = new ProductTable();
9         t.add("a", 10, 10);
10        t.add("b", 6, 10);

```

```

11         t.add("c", 2, 5);
12         t.add("d", 7, 110);
13
14         //
15
16         Map<Character, List<String>> grouped = t.nameStream(90)
17             .collect(Collectors.groupingBy(s -> s.charAt(0)));
18
19         System.out.println("                    : " + grouped);
20         System.out.println("
                                : " +
                                t.getProduct().map(p -> p.name).orElse("
                                "));
21     }
22 }

```

Листинг 4: Файл

```

1 package nd;
2
3 // simple class for 3d vector
4 public class Vector3D {
5     double x, y, z;
6
7     Vector3D(double x, double y, double z) {
8         this.x = x;
9         this.y = y;
10        this.z = z;
11    }
12
13    // method for cross product
14    public Vector3D cross(Vector3D other) {
15        double cx = y * other.z - z * other.y;
16        double cy = z * other.x - x * other.z;
17        double cz = x * other.y - y * other.x;
18        return new Vector3D(cx, cy, cz);
19    }
20
21    // method to calc length
22    public double length() {
23        return Math.sqrt(x * x + y * y + z * z);
24    }
25
26    public String toString() {
27        return "(" + x + ", " + y + ", " + z + ")";
28    }
29 }

```

Листинг 5: Файл

```
1 package nd;
2
3 import java.util.*;
4 import java.util.stream.*;
5 import java.util.Optional;
6
7 // class with methods for stream and optional
8 public class VectorOps {
9     List<Vector3D> vectors;
10
11     VectorOps(List<Vector3D> vectors) {
12         this.vectors = vectors;
13     }
14
15     // get stream of cross products of neighbors
16     public Stream<Vector3D> crossStream() {
17         return IntStream.range(0, vectors.size() - 1)
18             .mapToObj(i -> vectors.get(i).cross(vectors.get(i + 1)))
19     };
20
21     // find vector with max length
22     public Optional<Vector3D> maxVector() {
23         return crossStream()
24             .max(Comparator.comparingDouble(Vector3D::length));
25     }
26 }
```

Листинг 6: Файл

```
1 package nd;
2
3 import java.util.*;
4 import java.util.stream.*;
5 import static java.util.stream.Collectors.*;
6
7 public class Test {
8     public static void main(String[] args) {
9         // make list of vectors
10         List<Vector3D> list = Arrays.asList(
11             new Vector3D(1, 0, 0),
12             new Vector3D(0, 1, 0),
13             new Vector3D(0, 0, 1),
14             new Vector3D(1, 1, 1)
15         );
16     }
```

```

17     VectorOps ops = new VectorOps(list);
18
19     // group cross products by length < 1 or not
20     Map<String, List<Vector3D>> grouped = ops.crossStream()
21         .collect(groupingBy(v -> v.length() < 1 ? "short" : "
long"));
22
23     // print grouped
24     grouped.forEach((k, v) -> {
25         System.out.println(k + ":");
26         v.forEach(System.out::println);
27     });
28
29     // print max length vector
30     ops.maxVector().ifPresent(v -> {
31         System.out.println("max vector:");
32         System.out.println(v);
33     });
34 }
35 }

```

Листинг 7: Файл

```

1 package rd;
2
3 // class for interval (a, b)
4 public class Interval {
5     double a, b;
6
7     Interval(double a, double b) {
8         this.a = a;
9         this.b = b;
10    }
11
12    // check if this include other
13    public boolean includes(Interval other) {
14        return this.a <= other.a && this.b >= other.b;
15    }
16
17    // check if number inside
18    public boolean contains(double x) {
19        return a < x && x < b;
20    }
21
22    public String toString() {
23        return "(" + a + ", " + b + ")";
24    }

```

25 }

Листинг 8: Файл

```
1 package rd;
2
3 import java.util.*;
4 import java.util.stream.*;
5 import java.util.Optional;
6
7 // class with operations
8 public class IntervalList {
9     List<Interval> list;
10
11     IntervalList(List<Interval> list) {
12         this.list = list;
13     }
14
15     // make stream of intervals that include prev
16     public Stream<Interval> includedStream() {
17         return IntStream.range(1, list.size())
18             .mapToObj(i -> {
19                 Interval prev = list.get(i - 1);
20                 Interval curr = list.get(i);
21                 return curr.includes(prev) ? curr : null;
22             })
23             .filter(Objects::nonNull);
24     }
25
26     // find interval that include number x
27     public Optional<Interval> findContaining(double x) {
28         return list.stream()
29             .filter(it -> it.contains(x))
30             .findFirst();
31     }
32 }
```

Листинг 9: Файл

```
1 package rd;
2
3 import java.util.*;
4 import java.util.stream.*;
5 import static java.util.stream.Collectors.*;
6
7 public class Test {
8     public static void main(String[] args) {
```

```

9      // make list of intervals
10     List<Interval> data = Arrays.asList(
11         new Interval(-5.0, -1.0),
12         new Interval(-6.0, 0.0),
13         new Interval(-7.0, 1.0),
14         new Interval(0.0, 3.0),
15         new Interval(1.0, 5.0)
16     );
17
18     IntervalList ilist = new IntervalList(data);
19
20     // group by left and right borders
21     Map<String, List<Interval>> grouped = ilist.includedStream()
22         .collect(groupingBy(it -> {
23             if (it.b <= 0) return "right <= 0";
24             if (it.a < 0 && it.b > 0) return "cross zero";
25             return "left >= 0";
26         })));
27
28     // print groups
29     grouped.forEach((k, v) -> {
30         System.out.println(k + ":");
31         v.forEach(System.out::println);
32     });
33
34     // test find interval that include number
35     ilist.findContaining(2.5).ifPresentOrElse(
36         it -> System.out.println("found: " + it),
37         () -> System.out.println("not found")
38     );
39 }
40 }

```

Результат запуска представлен на рисунке 1.

Группировка: {a=[a], d=[d]}

Товар с наибольшим количеством: d

long:

(0.0, 0.0, 1.0)

(1.0, 0.0, 0.0)

(-1.0, 1.0, 0.0)

max vector:

(-1.0, 1.0, 0.0)

cross zero:

(-7.0, 1.0)

right <= 0:

(-6.0, 0.0)

found: (0.0, 3.0)

Рис. 1 — Результат